Vox: Technical Architecture Specification

Text-to-Speech Engine for AI Agents

July 2026 — v4.11.0

Contents

1 Overview **3**
- 1.1 Design Principles . . . 3
- 1.2 Component Map . . . 3

2 System Architecture **6**
- 2.1 Daemon and Connectivity . . . 6
- 2.2 Remote voxd . . . 7
- 2.3 System Paths . . . 7
- 2.4 Service Install and Identity . . . 7
- 2.5 Data Paths . . . 8
- 2.6 TTS Request Path (Text → Audio File → Playback) . . . 8
- 2.7 Notification Path (Hook Event → Audio) . . . 8
- 2.8 Session Lifecycle . . . 9

3 Provider Architecture **9**
- 3.1 Provider Protocol . . . 9
- 3.2 ElevenLabs . . . 10
- 3.3 OpenAI . . . 10
- 3.4 AWS Polly . . . 10
- 3.5 Offline Fallbacks . . . 10
- 3.6 Provider Selection . . . 10

4 MCP Tool Surface **11**
- 4.1 Background Synthesis Pattern . . . 12
- 4.2 Transport . . . 12

5 Audio Pipeline **12**
- 5.1 Text Splitting . . . 12
- 5.2 Segment Processing . . . 13
- 5.3 Audio Stitching . . . 13
- 5.4 Playback Queue . . . 13
- 5.5 Background Music . . . 13
- 5.6 Ephemeral Mode . . . 14
- 5.7 Format Handling . . . 14

6 CLI and Config **14**
- 6.1 CLI Commands . . . 14
- 6.2 Config File Format . . . 15
- 6.3 Credential Management . . . 16

7 Performance **16**
- 7.1 Latency Characteristics . . . 16
- 7.2 Caching . . . 17
- 7.3 Streaming vs. Batch . . . 17

8 Security **17**
8.1 Threat Model . . . 17
8.2 No Audio Until Opt-In . . . 17
8.3 Credential Isolation . . . 18

9 Platform Integration **18**
9.1 macOS . . . 18
9.2 Linux . . . 18

10 Plugin and Hook Integration **18**
10.1 Hook Registration . . . 18
10.2 Mute/Unmute Lifecycle . . . 19
10.3 Signal Classification . . . 19
10.4 Mood Inference . . . 20

11 Known Limitations **20**
11.1 No Streaming Playback . . . 20
11.2 No Intentional Audio Caching . . . 20
11.3 No Provider Failover . . . 20
11.4 No Playback Cancellation . . . 20
11.5 Background Failure Opacity . . . 21
11.6 Concurrent Config Writes . . . 21
11.7 NFS File Locking . . . 21
11.8 No Headless Audio Detection . . . 21
11.9 Voice List Staleness . . . 21

12 Test Architecture **22**

1 Overview

Vox is a text-to-speech engine with three projection surfaces from a single codebase: Python CLI, MCP server, and Claude Code plugin. It supports five synthesis backends (ElevenLabs, OpenAI, AWS Polly, macOS `say`, Linux `espeak-ng`) behind a provider-agnostic orchestration layer. Audio playback is serialized via file locking. Session state lives in a YAML frontmatter config file.

The system does *not* stream audio. Every synthesis call produces a complete MP3 file before playback begins. There is no audio mixing and no real-time pitch manipulation. Vox is a batch synthesizer with a playback queue. The `voxd` daemon accepts requests over WebSocket and serializes playback. Since v4.x it also generates, persists, and plays background music as an ownership-free *Program* — a named on-disk pool of instrumental tracks (`elevenlabs_music`) that fills in the background, auto-advances, and shuffle-rotates once full — through a separate reduced-volume subprocess outside the speech queue, and can be driven from a remote host over the network (`VOXD_HOST` / `VOXD_PORT` / `VOXD_TOKEN`).

1.1 Design Principles

P1. Provider-agnostic orchestration. The core module knows nothing about any specific TTS API. Providers implement a `TTSProvider` protocol; the core synthesizes, splits, stitches, and plays without knowing which backend produced the audio.

P2. No audio until opt-in. Vox is silent by default. The `notify` config field defaults to `"n"`. No hooks produce sound, no chimes play, and no synthesis calls fire until the user explicitly enables notifications via `/unmute` or `/vox`.

P3. Session-scoped state. Mutable state (voice, provider, mood) lives in the MCP server's memory, one instance per Claude Code session. Initial state is seeded from the per-repo split config (`.punt-labs/vox/vox.md` for durable preferences, `.punt-labs/vox/vox.local.md` for ephemeral session state) at startup; hook handlers also read and write these files for signal accumulation. The `voxd` daemon is stateless with respect to *speech* sessions—it synthesizes and plays whatever clients send it. Background music is the one piece of daemon-owned state: a single persisted, ownership-free *Program* that any session drives and no session owns (§5.5, DES-041).

P4. Fail fast. Invalid input raises exceptions. No defensive fallbacks, no silent degradation. If a voice doesn't exist, the call fails with a `VoiceNotFoundError`—it does not silently substitute another voice.

P5. Serialized playback. Concurrent synthesis calls never overlap audio output. The `voxd` daemon owns an in-memory playback queue with a single consumer—audio items play sequentially across all connected clients.

1.2 Component Map

`src/punt_vox/` holds roughly thirty modules plus three sub-packages (`src/punt_vox/voxd/`, `src/punt_vox/service/`, `src/punt_vox/providers/`). The daemon and the system-service installer were split from single files into packages as they grew. Domain types were split out of a single `types.py` into purpose-scoped modules. The tables below list the top-level modules first, then the package contents.

Top-level modules

Module	File	Responsibility
Types	`src/punt_vox/types.py`	Provider protocol and shared domain types (`TTSProvider`, `AudioProvider`, `MergeStrategy`)
Audio types	`src/punt_vox/types_audio.py`	Audio request/result value objects: `AudioRequest`, `AudioResult`, `HealthCheck`
Error types	`src/punt_vox/types_errors.py`	Domain error hierarchy (`VoiceNotFoundError` and siblings)
Synthesis types	`src/punt_vox/types_synthesis.py`	`SynthesisSpec` — validated voice-settings bundle passed to the daemon

Module	File	Responsibility
Core	`src/punt_vox/core.py`	`TTSClient`—provider-agnostic orchestration: text splitting, batch synthesis, pair stitching, audio merge
Config	`src/punt_vox/config.py`	`ConfigStore`/`VoxConfig`: read/write the split config (`vox.md` durable + `vox.local.md` ephemeral); `DURABLE_KEYS` / `EPHEMERAL_KEYS` routing
Dirs	`src/punt_vox/dirs.py`	Per-repo config-dir discovery (`find_config_dir`) and user-content dirs (`default_output_dir`, `music_output_dir`)
Paths	`src/punt_vox/paths.py`	*Authoritative* per-user state paths: `~/.punt-labs/vox` and its `logs/`, `run/`, `cache/` subdirs; `keys_env_file()`, `ensure_user_dirs()`
Resolve	`src/punt_vox/resolve.py`	Voice/language resolution, output directory resolution, vibe tag application
Output	`src/punt_vox/output.py`	Output path resolution: `VOX_OUTPUT_DIR` env, `~/Music/vox` fallback
Output fmt	`src/punt_vox/output_formatter.py`	Formats MCP tool output into the compact UI panel text the suppress-output hook emits
Playback	`src/punt_vox/playback.py`	`flock`-serialized playback: `play_audio()` (blocking), `enqueue()` (detached subprocess)
Hooks	`src/punt_vox/hooks.py`	Claude Code hook dispatchers: Stop, PostToolUse, Notification, PreCompact, UserPromptSubmit, SubagentStart/Stop, SessionEnd
Signal	`src/punt_vox/signal.py`	`Signal` / `SignalLog` domain extracted from `hooks.py`: classification and timestamped accumulation
Hook envelope	`src/punt_vox/hook_envelope.py`	Typed wrapper for a hook's JSON-in / JSON-out envelope
Hook payload	`src/punt_vox/hook_payload.py`	Parses the raw hook stdin payload into typed fields
Chime	`src/punt_vox/chime.py`	Mood-aware chime file selection
Music (CLI)	`src/punt_vox/cli_music.py`	Client-side `/music` command surface: on / off / play / list / next / loop and `playlist:N` part addressing
Program client	`src/punt_vox/program_gateway.py`	Translate music/Program commands into `voxd` Program RPCs over the WebSocket; thin, no business logic (with `src/punt_vox/program_control.py`)
Music prompts	`src/punt_vox/music_prompts.py`	Scaffolds the LLM-authored `base_prompt` plus twelve genre `variations` passed to `voxd` for track generation
Cache	`src/punt_vox/cache.py`	MP3 quip cache, content-addressed by (text, voice, provider)
Vibe	`src/punt_vox/vibe.py`	Vibe domain: mood text → ElevenLabs expressive-tag resolution, extracted from `hooks.py`
Voices	`src/punt_vox/voices.py`	Voice personality blurbs for roster display
Quips	`src/punt_vox/quips.py`	Immutable phrase pools for hook speech (stop, permission, idle, farewell, etc.)
Mood	`src/punt_vox/mood.py`	Mood family classification (bright/dark/neutral)
Normalize	`src/punt_vox/normalize.py`	Text normalization for speech (snake_case, camelCase, abbreviation expansion)

Module	File	Responsibility
Keys	`src/punt_vox/keys.py`	Provider API-key presence detection and reporting
API key resolver	`src/punt_vox/api_key_resolver.py`	`ApiKeyResolver` — resolves a per-call key from exactly one of four mutually-exclusive sources (`--api-key-file`, `--api-key-stdin`, `VOX_API_KEY`, `--api-key`)
Doctor	`src/punt_vox/doctor.py`	`vox doctor` health checks: Python, player binary, provider keys, daemon staleness
Daemon restarter	`src/punt_vox/daemon_restarter.py`	Detects a stale running `voxd` (version mismatch) and restarts it via the platform service backend
Desktop install	`src/punt_vox/desktop_install.py`	`DesktopInstaller` — registers the `vox` MCP server in Claude Desktop's config *without* embedding provider secrets; the daemon reads keys from `keys.env` at startup
Logging	`src/punt_vox/logging_config.py`	Rotating file log under `~/.punt-labs/vox/logs/`
Watcher	`src/punt_vox/watcher.py`	`SessionWatcher`—daemon thread tailing Claude session JSONL files for real-time signal classification and autonomous chime/speech in continuous mode
Client	`src/punt_vox/client.py`	WebSocket client for `voxd`: `VoxClient` (async) / `VoxClientSync` (sync), split across `src/punt_vox/client_gateway.py`, `src/punt_vox/client_env.py`, and `src/punt_vox/client_errors.py`. Lightweight—stdlib + websockets only
Server	`src/punt_vox/server.py`	FastMCP server (key: `mic`): the eleven MCP tools (see §4)
CLI	`src/punt_vox/__main__.py`	Typer CLI: unmute, record, vibe, music, notify, speak, voice, status, doctor, version, hook, daemon subcommands
Applet	`src/punt_vox/applet.py`	Lux display applet: element tree for the visual status surface

Sub-packages

Package	Module	Responsibility
`src/punt_vox/voxd/`	`daemon.py`	Daemon entry point; owns `DEFAULT_PORT=8421`, `DEFAULT_HOST`, lifecycle
	`router.py`	WebSocket message dispatch to handlers
	`speech_handlers.py`	synthesize / chime / record RPCs
	`system_handlers.py`	voices / health / control RPCs
	`synthesis.py`	Provider selection and synthesis pipeline
	`playback.py`	Daemon-side playback queue and consumer
	`dedup.py`	Content-addressed replay dedup with TTL
	`health.py`	Health-response assembly
	`chimes.py`	Chime asset resolution inside the daemon
	`config.py`	Daemon runtime config; `load_keys()` reads `keys.env` into the environment at startup
	`synthesis_result.py`	Typed synthesis outcome carried back across the WebSocket boundary

Package	Module	Responsibility
	`programs/`	Background-music *Program* package (ownership-free, persisted; §5.5): `program.py` / `state.py` / `invariants.py` (the state machine and its sixteen invariants); `control_channel.py` plus the typed `*_signal.py` (single-writer control); `filler.py` / `fill_reconciler.py` / `producer.py` / `retry_machine.py` (background fill and resilient retry); `rotate_policy.py` / `player.py` / `subprocess_player.py` (auto-advance, shuffle-rotation, reduced-volume playback); `filesystem_store.py` / `manifest.py` / `part_tags.py` (persistence and ID3 tagging); the per-command handlers; and `service.py` (composition root)
`src/punt_vox/service/`	`installer.py`	`ServiceInstaller` composing the platform backends; `install` / `uninstall` / health verification
	`launchd.py`	macOS `LaunchdBackend` — user LaunchAgent under `~/Library/LaunchAgents`
	`launchctl.py`	`LaunchctlAgent` — race-free launchd domain control (bootout-and-wait, then bootstrap with retry-on-error-5), shared by install and restart
	`systemd.py`	Linux `SystemdBackend` — unit under `/etc/systemd/system`
	`process.py`	`ProcessManager`: port-free enforcement, stale-daemon kill
	`keys_env.py`	`KeysEnvWriter`: writes provider keys to `keys.env`
`src/punt_vox/providers/`	`elevenlabs.py`	ElevenLabs TTS backend
	`elevenlabs_music.py`	ElevenLabs music-generation backend
	`openai.py`	OpenAI TTS backend
	`polly.py`	AWS Polly backend
	`say.py`	macOS `say` fallback
	`espeak.py`	Linux `espeak-ng` fallback
	`local_play.py`	Local-only playback provider
	`chunked.py`	Shared chunk-and-stitch helper
	`convert.py`	Format conversion (AIFF/WAV → MP3)
	`voice_resolver.py`	Shared voice-name resolution

2 System Architecture

2.1 Daemon and Connectivity

The system has two binaries from one package (`punt-vox`):

`voxd` System-level audio daemon. Listens on port 8421 via WebSocket. Owns synthesis (providers), playback queue, audio deduplication, and synthesis cache. Knows nothing about MCP, hooks, projects, or Claude Code.

`vox` Everything else: CLI commands, MCP server (`vox mcp`), hook handlers (`vox hook <event>`). All are WebSocket clients of `voxd`.

```
Claude Code <-- stdio --> vox mcp -- WebSocket --> voxd :8421
Hook scripts --> vox hook <event> -- WebSocket -->    |
Shell        --> vox unmute "hi"  -- WebSocket -->    |
                                                      v
                                                   speakers
```

The MCP session (Claude Code ↔ `vox mcp`) is stdio—unaffected by daemon restarts. The WebSocket connection (`vox mcp` ↔ `voxd`) reconnects automatically.

2.2 Remote voxd

By default `voxd` binds `127.0.0.1` and clients discover its port and token from the local `run/serve.{port,token}` files. DES-037 lets the client half run on a different machine from the daemon — the common case being SSH from a laptop with speakers to a headless server where synthesis and playback would otherwise be inaudible. Four environment variables configure it:

Var	Side	Default
`VOXD_HOST`	client	`127.0.0.1`
`VOXD_PORT`	client	from `serve.port`
`VOXD_TOKEN`	client	from `serve.token`
`VOXD_BIND`	server	`127.0.0.1`

Resolution is `explicit arg > env var > file > default`. Two deployment models are supported: direct network (same LAN) and SSH tunnel (different networks). The token is the security boundary and is redacted from access logs; there is no TLS on `voxd` itself. See `docs/remote-setup.md`.

2.3 System Paths

`voxd` runs as a single user, so *all* of its state lives under one per-user root: `~/.punt-labs/vox/`. The same scheme applies on macOS and Linux — there is no FHS split and no Homebrew prefix. `src/punt_vox/paths.py` is the single source of truth for these locations; `ensure_user_dirs()` creates the root and its `logs/`, `run/`, and `cache/` subdirs `chmod 0700` because each holds private per-user state (API keys, spoken-text logs, the auth token, cached audio).

An earlier release resolved these to FHS system paths (`/etc/vox`, `/var/log/vox`, `/var/run/vox`) on Linux and Homebrew-prefix paths on macOS. That was a regression: it stranded user API keys on upgrade, required `sudo` to edit personal tokens, and created a chown mismatch where root owned the file `voxd` was told to read. It has been removed.

Purpose	macOS	Linux
Keys	`~/.punt-labs/vox/keys.env`	
Cache	`~/.punt-labs/vox/cache/`	
Logs	`~/.punt-labs/vox/logs/voxd.log`	
Runtime	`~/.punt-labs/vox/run/serve.{port,token}`	
Service	`~/Library/LaunchAgents/com.punt-labs.voxd.plist`	`/etc/systemd/system/voxd.service`

The `run/` directory is created `chmod 0700` so other local users cannot read the auth token. Per-project enablement is a separate concern: the split config (`.punt-labs/vox/vox.md` + `vox.local.md`) is a client-side pair in the project directory. The daemon never reads it.

2.4 Service Install and Identity

`vox daemon install`

Runs as the invoking user. `install()` refuses to run with `os.geteuid() == 0`, so users who run `sudo vox daemon install` out of habit get an explicit error directing them to re-run without sudo. Per-user state under `~/.punt-labs/vox/` (keys, logs, runtime state, cache) is created with normal user permissions—no chown, no symlink defenses. `voxd` runs as the *installing user*, not root: it needs audio device access tied to the desktop session (CoreAudio on macOS, PulseAudio/PipeWire on Linux).

The two platforms differ in whether `sudo` is needed at all:

macOS — zero sudo (DES-038). Since v4.9.0 the daemon is a user *LaunchAgent* at `~/Library/LaunchAgents/com.p` not a root-owned LaunchDaemon. Both `install` and `uninstall` are fully sudo-free: `mkdir -p ~/Library/LaunchAgents`, write the plist, free the port, then `launchctl bootstrap gui/<uid>` and `launchctl kickstart`. The move fixed a 7x synthesis slowdown — LaunchDaemons run at background QoS with CPU demotion and I/O deprioritization; LaunchAgents run at user QoS. The plist drops `UserName` (invalid for agents) and adds `ProcessType=Interactive` to prevent App Nap throttling on the windowless daemon. There is no LaunchDaemon-to-LaunchAgent migration path — it was removed in v4.9.0 rather than shipped (only a handful of installs existed, and a compat shim would violate the no-backwards-compat-shim rule).

Linux — sudo for the unit file only. The systemd unit at `/etc/systemd/system/voxd.service` lives in a root-owned system directory, so placing it, reloading the manager, enabling it at boot, and cycling the process still escalate through `sudo`. The unit sets `User=` to the installing user so the daemon still runs unprivileged.

After registration, `install()` polls `voxd`'s health endpoint until it answers (5 s deadline) — `launchctl` / `systemctl` registration proves only that the job is scheduled, not that the daemon bound its port and stayed up. See DES-038 (and the superseded DES-029) in `DESIGN.md` for the full rationale.

2.5 Data Paths

Three primary data paths define the system's runtime behavior.

2.6 TTS Request Path (Text → Audio File → Playback)

1. Agent calls MCP tool (e.g., `unmute(text="Hello")`).
2. Server resolves provider: explicit parameter → session config → auto-detection (environment variable → API key probe → platform fallback).
3. Server resolves voice: explicit parameter → session config → language-aware default → provider default.
4. Server builds `AudioRequest` list (one per segment).
5. Server predicts output paths and returns immediately to the agent with predicted `AudioResult` metadata.
6. Background daemon thread: `TTSClient.synthesize_batch()` calls the provider's `synthesize()` for each request.
7. If multiple segments: `stitch_audio()` concatenates MP3 files with configurable inter-segment silence (default 500 ms) plus 150 ms trailing silence.
8. `enqueue(path)` copies the file to `~/.punt-labs/vox/pending/`, spawns a detached subprocess.
9. Subprocess acquires `flock(LOCK_EX)` on `~/.punt-labs/vox/playback.lock`, plays via `afplay` (macOS) or `ffplay` (cross-platform), deletes the pending copy on completion.

The agent receives the MCP response at step 5. Steps 6–9 happen asynchronously. The agent never blocks on synthesis or playback.

2.7 Notification Path (Hook Event → Audio)

1. Claude Code fires a lifecycle hook (Stop, Notification, PreCompact, etc.).
2. Shell script dispatcher (e.g., `hooks/notify.sh`) invokes `vox hook stop` with JSON on stdin.
3. Python hook handler reads session config, checks `notify` and `speak` fields.
4. If `notify` is `"n"`: no-op, immediate return.
5. If `speak` is `"n"` (chime mode): resolves mood-aware chime file, enqueues playback.

6. If `speak` is `"y"` (voice mode) and `vibe_signals` is non-empty: selects a random phrase from the relevant quip pool, calls `vox unmute` to synthesize and play. If `vibe_signals` is empty (no bash commands have run yet), the Stop hook is a no-op even with voice enabled.

7. For Stop hooks: returns {`"decision": "block", "reason": phrase`} to halt Claude's output while audio plays.

2.8 Session Lifecycle

1. **Session start.** `SessionStart` hook runs `session-start.sh`: deploys slash commands, cleans retired commands, auto-allows MCP tools. No audio plays.

2. **User enables voice.** `/unmute` or `/vox` sets `notify` to `"y"` or `"c"`. This is the first moment audio becomes possible.

3. **Signal accumulation.** Each `PostToolUse` (Bash) event classifies stdout against regex patterns and appends a timestamped signal (e.g., `tests-pass@10:45`) to `vibe_signals` in config.

4. **Mood inference.** On Stop hooks, the signal history is deterministically mapped to ElevenLabs expressive tags (e.g., `[relieved] [satisfied]`) without any LLM call.

5. **Session end.** `SessionEnd` hook speaks a farewell phrase and clears `vibe_signals`.

3 Provider Architecture

3.1 Provider Protocol

All providers implement `TTSProvider`, a Python `Protocol` class extending `AudioProvider`. Eleven required members:

Member	Signature and Purpose
`name`	`str` — Short identifier (e.g., `"elevenlabs"`)
`default_voice`	`str` — Fallback voice name
`supports_expressive_tags`	`bool` — Whether the provider interprets `[bracketed]` tags in text
`synthesize`	`(request, output_path) -> AudioResult` — Core synthesis
`resolve_voice`	`(name, language?) -> str` — Voice validation and normalization
`get_default_voice`	`(language) -> str` — Language-aware default
`list_voices`	`(language?) -> list[str]` — Available voices
`infer_language_from_voice`	`(voice) -> str | None` — Reverse lookup (best-effort)
`check_health`	`() -> list[HealthCheck]` — Provider-specific diagnostics
`generate_audio`	`(text, voice?, ...) -> AudioResult` — Convenience wrapper (inherited from `AudioProvider`)
`generate_audios`	`(texts, voice?, ...) -> list[AudioResult]` — Batch convenience wrapper

No provider SDK imports appear outside the provider's own file. Providers do import shared utilities from `core.py` (`split_text`, `stitch_audio`) for chunking, but the SDK libraries (`elevenlabs`, `openai`, `boto3`) are confined to their respective files.

3.2 ElevenLabs

Parameter	Value
Default model	`eleven_flash_v2_5` (low latency, 32 languages)
Expressive tags	Only on `eleven_v3`
Output format	MP3 44.1 kHz 128 kbps
Char limits	5 000–40 000 depending on model
Chunking	Auto-split at limit, stitch with 0 ms pause
Voice resolution	API fetch (cached globally), match by lowercase name or raw 20-char voice ID
Voice settings	stability, similarity_boost, style, speaker_boost (all optional floats 0.0–1.0)

Streaming: the SDK returns a generator of audio chunks. Vox iterates the generator to completion and writes the full file. There is no incremental playback—the entire response is buffered to disk before `enqueue()` is called.

3.3 OpenAI

Parameter	Value
Default model	`tts-1` (`tts-1-hd` available)
Expressive tags	Not supported
Max chars/call	4 096 (hard API limit)
Chunking	Auto-split at 4 096 chars, stitch
Voices	9 static: alloy, ash, coral, echo, fable, nova, onyx, sage, shimmer
Rate mapping	CLI percentage → OpenAI speed float (0.25–4.0, direct linear: rate/100)

3.4 AWS Polly

Parameter	Value
Output format	MP3
Engine preference	neural > generative > long-form > standard
Voice resolution	API fetch (paginated `DescribeVoices`, cached)
Language mapping	ISO 639-1 → BCP 47 (e.g., `de` → `de-DE`)
Auth	AWS credential chain (`aws sts get-caller-identity`)

Voice resolution bundles three Polly parameters (`voice_id`, `language_code`, `engine`) into a frozen `VoiceConfig` dataclass, so callers never need to know Polly's multi-parameter API.

3.5 Offline Fallbacks

Provider	Platform	Notes
`say`	macOS	System `say` command. Default voice: `samantha`. Output: AIFF converted to MP3 via ffmpeg. No API key required. No expressive tags.
`espeak`	Linux	`espeak-ng` command. Voice is the language code (e.g., `en`). No API key required. No expressive tags.

These exist so `vox doctor` and basic functionality work on machines with no cloud credentials. Audio quality is noticeably lower than cloud providers.

3.6 Provider Selection

Auto-detection follows a strict priority chain:

1. `TTS_PROVIDER` environment variable (explicit override).
2. `ELEVENLABS_API_KEY` present → `elevenlabs`.

3. `OPENAI_API_KEY` present → `openai`.
4. `aws sts get-caller-identity` succeeds → `polly`.
5. macOS with `say` binary → `say`.
6. Linux with `espeak-ng` or `espeak` → `espeak`.
7. Fallback: `polly` with a warning (will fail at synthesis time if no credentials).

Session-level overrides persist to the durable config (`.punt-labs/vox/vox.md`) when set via MCP tools. The `get_provider()` factory checks: explicit parameter → session config provider → auto-detect. Model resolution follows the same cascade: explicit → session config (only if provider matches) → `TTS_MODEL` env → provider default.

What this does not do: there is no failover. If the selected provider's API call fails, the error propagates. There is no automatic retry, no fallback to a cheaper provider, and no circuit breaker.

4 MCP Tool Surface

The MCP server registers as key `mic` and exposes eleven tools: four speech/state tools (`unmute`, `record`, `vibe`, `who`), three notification tools (`notify`, `speak`, `status`), and the four-tool music suite (`music`, `music_play`, `music_list`, `music_next`). All tools return JSON strings.

Tool	Signature and Purpose
`unmute`	`(text?, voice?, language?, segments?, rate=90, pause_ms=500, ephemeral=true, stability?, similarity?, style?, speaker_boost?, vibe_tags?, provider?, model?)` — Synthesize and play audio. `segments` is a list of {`text, voice?, language?, vibe_tags?`} dicts for multi-voice sequential playback. Returns immediately; synthesis and playback happen in a background daemon thread. Persists provider, model, and vibe_tags to session config.
`record`	`(text?, voice?, language?, segments?, rate=90, pause_ms=500, output_path?, output_dir?, stability?, similarity?, style?, speaker_boost?)` — Like `unmute` but writes to persistent path (default `~/Music/vox/`), no playback.
`vibe`	`(mood?, tags?, mode?)` — Set session mood, expressive tags, or vibe mode (`auto`/`manual`/`off`).
`who`	`(language?)` — List available voices: 6 random featured voices with personality blurbs, plus full roster. Filters by language if provided.
`notify`	`(mode?, voice?)` — Set notification mode: `y` (on-demand), `n` (off), `c` (continuous). First enable auto-sets `speak` to `"y"`.
`speak`	`(mode?, voice?)` — Set speech mode: `y` (voiced) or `n` (chimes only).
`status`	`()` — Returns full session state: provider, voice, notify, speak, vibe_mode, vibe, vibe_tags, vibe_signals, music_mode.

Tool	Signature and Purpose
`music`	`(mode, style?, name?, base_prompt?, variations?)` — Start (`"on"`) or stop (`"off"`) background music. When on, `voxd` builds (or reopens) a named, persisted *Program*: it plays the first generated track at once and fills a pool of up to twelve distinct instrumental tracks in the background, auto-advancing as each ends and shuffle-rotating the full pool at zero credits. The Program is daemon-owned and ownership-free — any session drives it (DES-041), superseding the old session-ownership model. A matching saved `name` reopens the pool with no generation.
`music_play`	`(name)` — Reopen and play a saved Program by name. No generation, no credits.
`music_list`	`()` — List saved Programs with name, size, and date.
`music_next`	`()` — Advance to the next track now: rotate the full pool (zero credits) or generate one if the pool is not yet full.

4.1 Background Synthesis Pattern

Both `unmute` and `record` return immediately with *predicted* results. The server computes deterministic output paths (MD5 hash of text → 12-hex-char filename) and returns metadata before synthesis begins. A daemon thread performs the actual provider API call and playback.

This means the agent sees the response in milliseconds, but audio may not play for several seconds (depending on provider latency). If synthesis fails in the background thread, the error is logged but not reported to the agent. The agent has no mechanism to detect synthesis failure after the tool call returns.

4.2 Transport

The MCP server runs on stdio transport (`stdin/stdout`). Logs go to stderr only. The server is started by Claude Code via `vox mcp` and communicates using the MCP JSON-RPC protocol. The MCP server delegates synthesis and playback to `voxd`, a persistent WebSocket daemon. Each Claude Code session spawns its own MCP server process, but all sessions share a single `voxd` instance.

5 Audio Pipeline

5.1 Text Splitting

`split_text(text, max_chars)` guarantees every chunk is $\leq$ `max_chars`:

1. If text fits: return as-is.
2. Split at sentence boundaries (regex `(?<=[.!?])\s+`).
3. Accumulate sentences into chunks up to the limit.
4. If a sentence exceeds the limit: split at word boundaries.
5. If a word exceeds the limit: split at character boundaries.

Whitespace between sentences is normalized to a single space. This function is provider-agnostic and used by both ElevenLabs (5 000–40 000 char models) and OpenAI (4 096 char hard limit).

5.2 Segment Processing

Multi-segment requests (via the `segments` parameter) are processed sequentially:

1. Each segment resolves its own voice and language independently, with per-segment overrides falling back to top-level defaults.
2. Per-segment `vibe_tags` override the top-level default (only for providers that support expressive tags).
3. Segments are synthesized to individual temp files.
4. `stitch_audio()` concatenates all segment files with `pause_ms` silence between each (default 500 ms).
5. 150 ms trailing silence is appended to prevent MP3 frame truncation artifacts.

5.3 Audio Stitching

`stitch_audio(segments, output_path, pause_ms)` uses pydub's `AudioSegment`:

- Loads each MP3 segment file.
- Inserts `AudioSegment.silent(duration=pause_ms)` between segments.
- Appends 150 ms trailing silence.
- Exports as MP3 to `output_path`.
- Raises `FileNotFoundError` if any segment is missing; `ValueError` if the segment list is empty.

pydub delegates to ffmpeg for MP3 decode/encode. This is the only place in the system that requires ffmpeg at runtime (playback uses `afplay` or `ffplay`, but stitching uses ffmpeg via pydub).

5.4 Playback Queue

Playback is serialized across all concurrent callers via POSIX file locking:

Component	Details
Lock file	`~/.punt-labs/vox/playback.lock`
Lock type	`fcntl.flock(fd, LOCK_EX)` (blocking)
Timeout	120 seconds
Player (macOS)	`afplay`
Player (other)	`ffplay -nodisp -autoexit -loglevel quiet`
Pending dir	`~/.punt-labs/vox/pending/`

`enqueue(path)` copies the audio file to the pending directory (preventing deletion races with ephemeral cleanup), then spawns a fully detached subprocess that acquires the lock, plays, and deletes the pending copy. The parent returns immediately.

What this does not do: there is no priority queue, no cancellation, no skip-ahead. If three audio files are enqueued, they play in FIFO order. There is no way to interrupt a playing file or flush the queue.

5.5 Background Music

Since v4.x, `voxd` generates, persists, and plays instrumental background music as an ownership-free *Program*: a named on-disk pool of tracks with an explicit, formally-modelled lifecycle (`docs/audio-programs.tex`; DES-041). This generalised and replaced an earlier session-owned, single-looping-track design — the superseded DES-031 (session ownership) and DES-033 (vibe-triggered regeneration). Music deliberately sidesteps the speech playback queue:

- **Ownership-free, daemon-owned (DES-041).** `voxd` holds exactly one Program. There is no `owner_id` and no non-owner rejection — any session, and the CLI, drive the same Program. A single-writer `ControlChannel` serialises every mutation as a typed `ControlSignal`, so concurrent clients never race the pool, and `ProgramState` re-validates all sixteen state invariants by construction on every transition.
- **Pool, auto-advance, and rotation.** `music("on")` plays the first generated track immediately, then a background filler generates up to twelve distinct tracks (one `asyncio` task at a time). Playback auto-advances as each track ends; once the pool is full, generation stops and playback shuffle-rotates the pool — never repeating the just-played track — at zero credits. A resilient retry sub-machine (`retry_machine.py`) absorbs transient generation failures without corrupting the pool (see the `RetryCapped`/`RetryExhausted` transitions in the Z model).
- **Persisted and replayable.** Each Program is a directory `~/Music/vox/<name>/` of `NNN.mp3` parts plus a manifest; every track carries ID3 tags (artist `vox`, album = program name, title = the variation clause, genre = style, and track index), so pools drop straight into Music.app or Ubuntu players. Both the CLI and MCP can `list`, `play`, `loop`, and address a specific part (`playlist:N`) of a saved Program — consume-only replay with no generation and no credits.
- **Separate subprocess at reduced volume (DES-030).** Each track plays through its own player process (`afplay --volume 0.3` on macOS, `ffplay -volume 30` on Linux), completely outside the speech mutex. Speech and chimes overlay at full volume; the volume differential makes speech intelligible over the music with no runtime ducking or pausing.
- **LLM-authored prompts.** The agent authors a `base_prompt` plus twelve genre-accurate `variations` (scaffolded by `music_prompts.py`); `voxd` is a pipe to ElevenLabs and never decides what a genre sounds like. With no prompts supplied it falls back to a bare stylistic default.

5.6 Ephemeral Mode

When `ephemeral=true` (the default for `unmute`), output is ephemeral: previous ephemeral files are cleaned up before each synthesis. In the current daemon-centric design `voxd` owns ephemeral cleanup internally (writing under `.punt-labs/vox/ephemeral/`); the `ephemeral` flag is still accepted on the tool surface for callers.

Persistent output (via `record` or `ephemeral=false`) goes to `~/Music/vox/` (overridable via `VOX_OUTPUT_DIR`) or a user-specified directory. Generated music Programs live under `~/Music/vox/<name>/` — one directory per named Program (§5.5).

5.7 Format Handling

All providers output MP3. Cloud providers (ElevenLabs, OpenAI, Polly) produce MP3 natively. The `say` provider produces AIFF via the macOS `say` command, then converts to MP3 via ffmpeg. The `espeak` provider writes WAV via `espeak-ng`, then converts to MP3 via ffmpeg. This means all providers produce files that pydub can decode, and multi-segment stitching works across all backends when ffmpeg is available.

6 CLI and Config

6.1 CLI Commands

Command	Purpose
`vox unmute [TEXT]`	Synthesize and play. `--from FILE` for batch input (JSON array). Ephemeral output to `.vox/`.
`vox record [TEXT]`	Synthesize and save. `--output PATH` or `--output-dir DIR`. No playback.
`vox vibe MOOD`	Set session mood. Special values: `auto` (infer from signals), `off` (disable).

Command	Purpose
`vox music [on|off]`	Start/stop background music. `--style` (e.g. `techno`), `--name` to save or replay a track.
`vox notify [y|n|c]`	Set notification mode. `y` = on-demand, `n` = off, `c` = continuous. Optional `--voice`.
`vox speak [y|n]`	`y` = voiced, `n` = chimes only.
`vox voice NAME`	Set session voice.
`vox status`	Show session state (provider, voice, mood, signals).
`vox doctor`	Health checks: Python version, player binary, provider API keys, provider-specific checks.
`vox version`	Print version string.
`vox mcp`	Start MCP server on stdio.
`vox daemon <cmd>`	Manage the `voxd` system service: `install`, `uninstall`, `status`, `restart`.
`vox hook <event>`	Dispatch hook handler (stop, post-bash, notification, pre-compact, user-prompt-submit, subagent-start, subagent-stop, session-end).

Global flags: `--json` (JSON output), `--verbose` (debug logging), `--quiet` (suppress non-JSON output). `--verbose` and `--quiet` are mutually exclusive.

6.2 Config File Format

Session state is a *split* pair of YAML-frontmatter files under `.punt-labs/vox/` in the repo root (DES-036). Durable preferences are tracked in git; ephemeral session state is gitignored. Which file a field lives in is decided by the `DURABLE_KEYS` / `EPHEMERAL_KEYS` frozensets in `src/punt_vox/config.py`.

```
# .punt-labs/vox/vox.md (durable, tracked)
---
notify: "y"
speak: "y"
voice: "matilda"
provider: "elevenlabs"
model: "eleven_v3"
vibe_mode: "auto"
---

# .punt-labs/vox/vox.local.md (ephemeral, gitignored)
---
vibe: "focused coding session"
vibe_tags: "[calm] [focused]"
vibe_signals: "tests-pass@10:45,git-commit@10:47"
---
```

Field	File	Values	Purpose
`notify`	`vox.md`	`y`, `n`, `c`	Notification mode
`speak`	`vox.md`	`y`, `n`	Voice vs. chime mode
`voice`	`vox.md`	string or null	Session voice name
`provider`	`vox.md`	string or null	TTS provider override
`model`	`vox.md`	string or null	TTS model override
`vibe_mode`	`vox.md`	`auto/manual/off`	Mood tracking mode
`vibe`	`vox.local.md`	string or null	Human-readable mood
`vibe_tags`	`vox.local.md`	string or null	ElevenLabs expressive tags
`vibe_signals`	`vox.local.md`	string or null	Timestamped signal history

`ConfigStore` owns a config directory (default `.punt-labs/vox/`); `find_config_dir()` walks up from cwd looking for a directory containing `vox.md` or `vox.local.md` (no legacy `.vox/` fallback). Reads merge the two files — durable as the base layer, ephemeral overlaid but accepting only `EPHEMERAL_KEYS`.

`write_field()` and `write_fields()` validate keys against `ALLOWED_CONFIG_KEYS` before writing and route each key to the correct file; unknown keys raise `ValueError`, and values containing newlines are rejected to protect the frontmatter. If the target file does not exist, a minimal file is created with YAML frontmatter delimiters.

6.3 Credential Management

Provider API keys are read from environment variables:

Variable	Provider
`ELEVENLABS_API_KEY`	ElevenLabs (TTS + music)
`OPENAI_API_KEY`	OpenAI
AWS credential chain	Polly (via boto3)

Because `voxd` runs as a detached service, it cannot inherit the shell environment of the session that installed it. At `vox daemon install`, `KeysEnvWriter` snapshots the relevant provider keys from the installing environment into `~/.punt-labs/vox/keys.env` (`chmod 0700` directory) so the daemon can read them at startup. There is no keychain integration and no encrypted storage. `vox doctor` checks for the presence of these variables and reports their status.

Per-call API keys

The CLI also accepts a *per-call* key that overrides the ambient credentials for a single invocation — for billing a synthesis to a specific account. `ApiKeyResolver` resolves it from exactly one of four mutually-exclusive sources, rejecting any combination:

1. `--api-key-file <path>` (mode checked; warns if broader than `0600`).
2. `--api-key-stdin` (one line piped in; refuses a tty).
3. `VOX_API_KEY` environment variable.
4. `--api-key <value>` on argv (warns that argv is visible via `ps` and shell history).

Absent all four, the call uses the ambient provider credentials.

Remote daemon auth

When a client talks to a remote `voxd` (DES-037), the auth token is the security boundary. It resolves `explicit arg > VOXD_TOKEN env > serve.token file`, and access logs redact it. `VOXD_HOST` and `VOXD_PORT` select the remote endpoint (default `127.0.0.1` and the local `serve.port`); `VOXD_BIND` sets the daemon's bind address server-side (default `127.0.0.1`).

7 Performance

7.1 Latency Characteristics

Provider	Typical Latency	Notes
ElevenLabs (flash)	200–800 ms	Streaming API, buffered to disk
ElevenLabs (v3)	500–2 000 ms	Higher quality, higher latency
OpenAI (tts-1)	300–1 000 ms	No streaming, single HTTP call
OpenAI (tts-1-hd)	500–1 500 ms	Higher quality variant
Polly (neural)	200–600 ms	AWS region-dependent
macOS `say`	50–200 ms	Local, no network
`espeak-ng`	20–100 ms	Local, no network

These are synthesis latencies (time to first byte from provider). Total time-to-audio includes stitching (10–50 ms for pydub operations) and playback queue wait (0 ms if queue is empty, unbounded if other audio is playing).

7.2 Caching

There is no intentional audio caching. Deterministic filenames (MD5 hash of text) mean that repeated calls for the same text produce the same output path. The MCP server's `_synthesize_segments()` skips synthesis when the output file already exists, which provides incidental caching for non-ephemeral calls. In ephemeral mode, `clean_ephemeral()` deletes prior MP3 files before each call, so the skip rarely triggers.

Voice lists are cached globally per provider (populated on first call, never refreshed within a process). Config path resolution is cached via `lru_cache(maxsize=1)`.

7.3 Streaming vs. Batch

ElevenLabs returns a streaming response (chunk generator). Vox iterates the generator to completion and writes the full file before playback. This adds latency equal to the full synthesis time but simplifies the pipeline: every provider produces a complete file, and the playback system has a single code path.

What this means: for a 30-second audio clip, the user waits for the full 30 seconds of synthesis before hearing anything. True streaming playback (play while synthesizing) is not implemented.

8 Security

8.1 Threat Model

Vox runs in the user's process space with the user's credentials. The primary trust boundary is the MCP tool interface: Claude Code can call any tool, but tools only perform synthesis and configuration—no file deletion, no network access beyond provider APIs, no shell execution.

Threat	Description	Mitigation
API key exposure	Provider keys in env vars visible to any process in the session.	Standard env var practice. No logging of key values. `doctor` reports presence, not content.
Unintended audio	Audio plays without user consent, e.g., during a meeting.	Default `notify=n`. No audio until explicit opt-in. `/mute` immediately silences.
Cost accumulation	Rapid synthesis calls exhaust provider quotas.	No rate limiting in Vox. Provider-side quotas are the only protection.
Pending file race	Concurrent processes write to `pending/` with colliding filenames.	Filenames include PID prefix. `flock` serializes playback.

8.2 No Audio Until Opt-In

This is the system's primary safety property. On fresh install:

- `.vox/config.md` does not exist.
- `notify` defaults to `"n"`.
- All hook handlers check `notify` first and return immediately if `"n"`.
- The `SessionStart` hook deploys commands but produces no audio.
- No MCP tool produces audio unless the agent explicitly calls `unmute`.

The user must take an affirmative action (`/unmute`, `/vox y`, or the agent calling `notify(mode="y")`) to enable any audio output.

8.3 Credential Isolation

Each provider file is the only file that imports its SDK:

- `src/punt_vox/providers/elevenlabs.py` — only file importing `elevenlabs`
- `src/punt_vox/providers/openai.py` — only file importing `openai`
- `src/punt_vox/providers/polly.py` — only file importing `boto3`

API clients are instantiated inside provider constructors. No client objects are passed between modules. No credentials are logged at any level.

9 Platform Integration

9.1 macOS

- **Playback.** `afplay` is the primary player, available on all macOS installations. `ffplay` is the fallback (requires Homebrew ffmpeg).
- **Offline fallback.** The `say` provider uses the system's built-in speech synthesis. Voice names are macOS-specific (e.g., `samantha`, `alex`).
- **File locking.** `fcntl.flock()` works correctly on APFS and HFS+.
- **Temp directory.** `.envrc` sets `TMPDIR` to `.tmp/` in the project root, keeping temp files workspace-local.

9.2 Linux

- **Playback.** `ffplay` from the `ffmpeg` package. No `afplay` equivalent. `resolve_player()` raises `FileNotFoundError` if neither is available.
- **Offline fallback.** The `espeak` provider wraps `espeak-ng` (preferred) or `espeak`.
- **File locking.** `fcntl.flock()` works on ext4, btrfs, and most Linux filesystems. NFS mounts may not honor `flock`; this is not handled.
- **No headless mode.** Playback requires an audio device. Running in a headless container or SSH session without PulseAudio/ALSA will cause `ffplay` to fail. There is no detection or graceful degradation for this case.

10 Plugin and Hook Integration

Vox integrates with Claude Code through a plugin manifest (`plugin.json`) and a hook registration file (`hooks.json`). The plugin registers an MCP server (key: `mic`) and nine lifecycle hooks.

10.1 Hook Registration

Event	Script	Behavior
`SessionStart`	`session-start.sh`	Deploy slash commands, clean retired commands, auto-allow MCP tools. No audio.
`PostToolUse` (MCP)	`suppress-output.sh`	Format MCP tool output as compact UI panel text.

Event	Script	Behavior
PostToolUse (Bash)	signal.sh	Classify stdout, append signal to vibe_signals.
Stop	notify.sh	Block with speech phrase or play chime. Returns {decision: "block"} to pause Claude while audio plays.
Notification	notify-permission.sh	Play chime or speak permission/idle phrase (async).
PreCompact	pre-compact.sh	Speak "grabbing a snack" phrase (continuous mode only, async).
UserPromptSubmit	acknowledge.sh	Speak acknowledgment phrase (continuous mode only, async).
SubagentStart/Stop	subagent.sh	Announce subagent spawn/completion (continuous mode only, async).
SessionEnd	farewell.sh	Speak farewell, clear vibe_signals (async).

10.2 Mute/Unmute Lifecycle

Three notification states control audio behavior:

State	Hooks Fire?	Audio?
notify=n	All hooks return immediately	No audio from hooks. Agent can still call unmute directly.
notify=y	Stop, Notification, SessionEnd	Chimes (speak=n) or speech (speak=y) on task completion and permission requests.
notify=c	All hooks fire	Continuous narration: acknowledgments, subagent announcements, pre-compact phrases, farewells.

speak is orthogonal to notify:

- speak=y: hooks synthesize speech via provider.
- speak=n: hooks play pre-recorded chime MP3 files from assets/ (no provider API call).

10.3 Signal Classification

Signal classification uses a single canonical pattern table in hooks.py. The PostToolUse (Bash) hook calls classify_signal() directly. The session watcher in watcher.py delegates to the same function via classify_output(), passing exit_code=None. The eight regex-based signal patterns are:

Signal	Detection Patterns
lint-fail	Found [0-9]+ error
lint-pass	All checks passed, 0 errors
tests-pass	[0-9]+ passed, tests? ok, ✓.*passed
tests-fail	FAILED, AssertionError, ERRORS?\b
merge-conflict	CONFLICT
git-push-ok	Everything up-to-date, ->.*main
git-commit	^\[.+\] .+, ^create mode
pr-created	pull/[0-9]+, created pull request

If no pattern matches but exit code is non-zero, the signal is `cmd-fail`. Signals are appended as {signal}@{HH:MM} to the comma-separated `vibe_signals` config field.

10.4 Mood Inference

On Stop hooks, accumulated signals are deterministically mapped to ElevenLabs expressive tags. No LLM call is involved:

Signal Trajectory	Tags
Recovery (fail → pass)	[relieved]
Shipped, no failures	[satisfied]
Shipped after failures	[relieved] [satisfied]
Ended with failure, many fails	[frustrated] [sighs]
Many passes, no failures	[excited]
Some passes	[calm]
Default	[calm]

This only affects audio when the provider supports expressive tags (`eleven_v3` model only). All other providers ignore the tags silently.

11 Known Limitations

11.1 No Streaming Playback

Audio is fully synthesized to disk before playback begins. For long texts, this means the user waits for the entire synthesis to complete. ElevenLabs returns a streaming response, but Vox buffers it entirely. True streaming playback (play the first chunk while subsequent chunks synthesize) is not implemented.

11.2 No Intentional Audio Caching

The MCP server skips synthesis when the output file already exists at the deterministic path, but ephemeral cleanup deletes prior files before most calls. There is no cache layer, no TTL, and no cache invalidation. For non-ephemeral use patterns with repetitive text, the file-exists check provides incidental reuse. For ephemeral calls (the common path), every synthesis hits the provider API.

11.3 No Provider Failover

If the selected provider fails (API error, rate limit, credentials expired), the error propagates to the caller. There is no automatic fallback to a lower-tier provider, no retry with exponential backoff, and no circuit breaker.

11.4 No Playback Cancellation

Once audio is enqueued, it plays to completion. There is no skip, pause, cancel, or flush-queue mechanism. If three long audio clips are enqueued, they play sequentially with no way to interrupt.

11.5 Background Failure Opacity

The `unmute` and `record` MCP tools return predicted results before synthesis completes. If synthesis fails in the background thread (provider error, network timeout, disk full), the error is logged to stderr but not reported to the agent. The agent has no mechanism to detect or recover from synthesis failures after the tool call returns. The session watcher's autonomous speech also fails silently (bare `except Exception` with logging).

11.6 Concurrent Config Writes

`write_fields()` in `config.py` performs a read-then-write with no file-level lock. The MCP server's background synthesis thread and `PostToolUse` hook handlers can write `vibe_signals` concurrently. Both paths do unguarded read-modify-write on the same file. The race window is narrow but real in continuous mode with fast bash commands. The playback lock (`playback.lock`) governs audio playback, not config writes.

11.7 NFS File Locking

The playback serialization relies on `fcntl.flock()`. On NFS mounts, `flock` may not provide mutual exclusion. If `~/.punt-labs/vox/` is on an NFS filesystem, concurrent playback from multiple machines could overlap.

11.8 No Headless Audio Detection

On Linux, if no audio output device is available (headless container, SSH without PulseAudio), `ffplay` fails silently or with an error. Vox does not detect this condition and does not warn the user.

11.9 Voice List Staleness

Provider voice lists are cached globally on first access and never refreshed within a process lifetime. If a voice is added or removed on the provider side, the change is invisible until the process restarts (next Claude Code session).

12 Test Architecture

Test File	What It Tests
`test_types.py`	Domain type validation, `generate_filename()` determinism
`test_core.py`	Text splitting edge cases, `TTSClient` batch/pair synthesis, stitching
`test_config.py`	Config read/write, worktree resolution, field validation
`test_output.py`	Output path resolution, env var handling
`test_normalize.py`	Text normalization for speech
`test_cache.py`	MP3 quip cache, content addressing
`test_playback.py`	`flock` serialization, `enqueue()` subprocess lifecycle
`test_hooks.py`	Signal classification, mood inference, hook handler dispatch
`test_cli.py`	CLI command smoke tests
`test_server.py`	MCP tool behavior with mocked providers
`test_elevenlabs_provider.py`	Voice resolution, chunking, voice settings, health checks
`test_openai_provider.py`	Rate mapping, chunking at 4 096 chars
`test_polly_provider.py`	VoiceConfig bundling, engine preference, Polly API mocks
`test_say_provider.py`	macOS say command construction
`test_espeak_provider.py`	espeak-ng command construction
`test_client.py`	`VoxClient`/`VoxClientSync` WebSocket client for `voxd`
`test_service_*.py`	Service lifecycle, split per backend: `installer`, `launchd`, `systemd`, `process`, `keys_env`
`test_signal.py`	`Signal`/`SignalLog` classification and accumulation
`cli_music.py` tests, `tests/programs/`	The client-side music command surface and the `voxd/programs/` Program suite: state-machine invariants (`test_invariants`, `test_properties`), the single-writer control channel, the filler and retry sub-machines, rotation, the filesystem store and manifest, and the per-command handlers
`test_api_key_resolver.py`	Four-source mutual-exclusion key resolution and permission warnings
`test_paths.py`	Per-user path resolution and `ensure_user_dirs` permissions
`test_doctor.py`, `test_daemon_restarter.py`	Health checks and stale-daemon restart

The table above is representative, not exhaustive: the suite mirrors every source module, including the split domain types (`test_types_synthesis.py`), the hook envelope/payload modules, the output formatter, and each provider (including `elevenlabs_music` and `voice_resolver`).

Quality gates: `ruff` (lint + format), `mypy` (strict), `pyright` (strict), `pytest`, `shellcheck` (all shell scripts in `hooks/` and `scripts/`). Zero violations required.

Polly test fixtures use `AudioSegment.silent(duration=50)` to generate valid MP3 bytes, because pydub delegates to ffmpeg which rejects fake byte sequences. Provider mocks use `side_effect=lambda` rather than `return_value` to produce fresh mock objects per call.